

This homework has 8 questions, for a total of 100 points and 3 extra-credit points.

Unless otherwise stated, each question requires *clear*, *logically correct*, and *sufficient* justification to convince the reader.

We strongly encourage you to typeset your solutions in L^AT_EX.

If you collaborated with someone, you must state their name(s). You must *write your own solution* for all problems and *may not use any other student's write-up or any AI tools*.

0. **Before you start; before you submit.**

If applicable, state the name(s) and username(s) of your collaborator(s).

Solution:

- (10 pts) 1. **Self assessment.** Carefully read and understand the posted solutions to the previous homework. Identify one part for which your own solution has the most room for improvement (e.g., has unsound reasoning, doesn't show what was required, could be significantly clearer or better organized, etc.). Copy or screenshot this solution, then in a few sentences, explain what was deficient and how it could be fixed.

(Alternatively, if you think one of your solutions is significantly *better* than the posted one, copy it here and explain why you think it is better.)

If you didn't do last week's homework, choose a problem from it that looks challenging to you, and in a few sentences, explain the key ideas behind its solution in your own words.

Solution:

2. **Fixed-color-sequence paths.**

Let $G = (V, E)$ be an undirected graph with n vertices and m edges, where $m \geq n$. Also, suppose that each edge is colored either black or white. We say that a path $P = (e_1, \dots, e_k)$ of edges has *color sequence* $(c_1, \dots, c_k)$, where each $c_i \in \{\text{black}, \text{white}\}$, if each e_i has color c_i . In this problem, you will design an algorithm that, given two vertices s, t and a desired color sequence $(c_1, \dots, c_k)$, determines in $O(mk)$ time whether there exists a path of length k from s to t with that color sequence. Note that paths in this problem do not need to be *simple*; they can visit vertices or edges more than once.

The graph $G = (V, E)$ is represented via *adjacency lists*, i.e., for each vertex $i \in V$ there is a list of all its neighbors: vertices j such that $(i, j) \in E$.

- (10 pts) (a) Derive a recurrence relation, including base case(s), for this problem. Briefly justify its correctness.

Solution: Define $P^{(i)}(v)$ to denote whether there exists a path from s to v in G with color sequence $(c_1, \dots, c_i)$. So our goal is to compute $P^{(k)}(t)$. Observe that we have the following recurrence for $P^{(i)}(v)$:

$$P^{(i)}(v) = \bigvee_{(u,v) \in E: \text{color}(u,v)=c_i} P^{(i-1)}(u)$$

with the base case

$$P^{(0)}(v) = \begin{cases} \text{True} & \text{if } v = s \\ \text{False} & \text{if } v \neq s. \end{cases}$$

This is because there is a path from s to v with color sequence $(c_1, \dots, c_i)$ if and only if, for some incoming neighbor u of v , there is path from s to u with color sequence $(c_1, \dots, c_{i-1})$ and the color of (u, v) is c_i . The base case is straightforward: since $i = 0$, only the empty path (of no edges) has an empty (length-0) color sequence, so the base case is true if and only if the starting and ending vertices are equal.

-
- +2 For correctly giving the base case, including both $v = s$ and $v \neq s$, with justification.
 - +2 For correctly identifying that this is a 2D DP problem.
 - +1 For checking all edges incident to vertex v .
 - +2 For stating the correct recurrence, penalize 1 points for mistakes in writing the recurrence if the justification is correct.
 - +3 Giving correct justification for the recurrence.

- (6 pts) (b) Give pseudocode implementing an $O(km)$ -time dynamic programming solution to this problem, and justify its running time. You do not need to give a correctness analysis since you did so for your recurrence.

Solution: The recurrence directly suggests the following bottom-up dynamic programming algorithm. It iterates from $i = 1$ to k , so every cell that is read from the table has already been computed when it is needed.

```
function COLORSEQUENCEPATH( $G = (V, E), s, t, (c_1, \dots, c_k)$ )
   $\forall v \in V$  table(0,  $v$ )  $\leftarrow$   $\begin{cases} \text{True} & \text{if } v = s \\ \text{False} & \text{if } v \neq s. \end{cases}$ 
  for  $i = 1$  to  $k$  do
    for each vertex  $v \in V$  do
      table( $i, v$ ) =  $\bigvee_{(u,v) \in E: \text{color}(u,v)=c_i}$  table( $i - 1, u$ )
  return table( $k, t$ ).
```

The running time is $O(km)$ because there are k iterations of the outer loop, and each iteration looks at each vertex exactly once and each edge exactly twice (once for each endpoint of the edge, since the graph is undirected), doing constant work for each.

-
- +1 Returns the correct answer.
 - +1 Mentions base case correctly in the pseudocode

- +1 For filling the table in the correct order. The table should be filled for each vertex at each value of $i = 0, \dots, k$ before moving to the next value of i .
- +1.5 For truthful implementation of their recurrence. It's ok if they copy the expression for the recurrence into the pseudocode instead of explicitly iterating through all of the incident edges in the for loop.
- +1.5 For correct analysis of running time explaining how they get $O(km)$.

3. **Back to college** Michael drives down to Ann Arbor from his hometown for the start of each school year. Interested in minimizing his commute time, he invents a modular fuel pack for his car for the commute through various towns in Michigan. This fuel pack allows the vehicle to travel a certain distance before refueling. Each town has a service station where a fuel pack can be reloaded: the current pack is simply swapped out for a fresh full one. (Any remaining fuel in the removed pack will not be used by the car.) The goal is to visit the towns in a specified order, while minimizing the number of fuel pack reloads.

We model this problem as follows. The car can travel a distance up to R on a single fuel pack. Starting from town 0, there are n other towns to visit in sequence. For $i = 1, \dots, n$, the distance from the $(i - 1)$ st town to the i th town is $d_i \leq R$. Given R and an array $D = [d_1, \dots, d_n]$ as input, the goal is to find a *smallest* set $S \subseteq \{1, \dots, n\}$ of towns (specified by their indices) where refueling should occur, so that the car can reach all n towns without running out of fuel at any point. The car starts out with a full fuel pack, and no refueling is needed at the final town.

- (7 pts) (a) Give a greedy algorithm (including pseudocode) that solves this problem. Defer a correctness argument to the next part, but provide everything else involved in “giving an algorithm” here.

Solution: Consider the starting point as town $i = 0$. For convenience in the pseudocode, also define the ‘sentinel’ value $D[n + 1] = \infty$.

The algorithm repeats the following: from the current town i , find the furthest town $k > i$ such that the distance $\sum_{j=i+1}^k D[j]$ from town i to town k is at most R . Refuel at town k if $k < n$ and repeat the process (if $k = n$, the trip is complete), now using k as the current town.

```
1: function GREEDY( $D[1, \dots, n], R$ )
2:    $i \leftarrow 0, S \leftarrow []$  //  $S$  is the list of towns where we refuel
3:   while  $i < n$  do
4:      $j \leftarrow i, t \leftarrow 0$  //  $t$  is distance from town  $i$  to  $j$ 
5:     while  $t \leq R$  do
6:        $j \leftarrow j + 1$ 
7:        $t \leftarrow t + D[j]$  // will be  $\infty$  for  $j = n + 1$ 
8:     if  $j - 1 < n$  then
9:        $S \leftarrow \text{append}(S, j - 1)$ 
10:     $i \leftarrow j - 1$ 
11:  return  $S$ 
```

Since no town is farther than R from the previous one, we strictly increase i in each iteration of the outer loop, ensuring that we access each element of the array at most twice. Hence, there are at most n iterations of the outer loop, and at most $2n$ iterations *in total* of the inner loop. Therefore, the overall running time of the algorithm is $O(n)$.

+4 Correct algorithm

+3 Correct runtime analysis

- (10 pts) (b) The setup for your correctness proof is as follows. Let OPT be some arbitrary optimal solution. Let $\text{ALG} = i_1, i_2, i_3, \dots, i_k$ be the towns chosen by your algorithm, in their greedily chosen order. If $\text{ALG} = \text{OPT}$, then ALG is optimal, as needed. So suppose that $\text{ALG} \neq \text{OPT}$, and consider the *first* town i_{diff} in ALG that is not in OPT. (Such an i_{diff} must exist because ALG must have at least as many towns as OPT, since OPT is optimal.) So, OPT contains all of $i_1, i_2, i_3, \dots, i_{\text{diff}-1}$, but not i_{diff} . You will construct another optimal solution OPT' that contains all of $i_1, i_2, i_3, \dots, i_{\text{diff}}$ by modifying OPT, exchanging out some suitable town j for i_{diff} instead. Describe which town j to exchange for i_{diff} , and prove that OPT' is an optimal solution. In particular, you should prove that OPT' is a valid solution *and* an optimal solution.

Solution: Let j be the next town after $i_{\text{diff}-1}$, at which OPT refuels. Note that such a j must exist, otherwise OPT could reach the final town directly from $i_{\text{diff}-1}$, so ALG would also finish the trip without choosing i_{diff} , a contradiction. (It would also be valid to let j be the first town in $\text{OPT} \setminus \text{ALG}$.)

First we will establish that $j < i_{\text{diff}}$. Since OPT is a valid solution, the distance from town $i_{\text{diff}-1}$ to town j is at most R . The algorithm defines i_{diff} as the *furthest* town such that the distance between $i_{\text{diff}-1}$ and i_{diff} is at most R , which means that $j < i_{\text{diff}}$. Next, we will show that OPT' is a valid solution. First we will show that the distance from i_{diff} to the next town of OPT' , is at most R . Because town j was the only town exchanged out of OPT to form OPT' , and $j < i_{\text{diff}}$, we know that OPT' contains *all* towns of OPT that are after i_{diff} . Since OPT is a valid solution, the distance from *any* town (in particular i_{diff}) to next town of OPT is at most R . Thus, the distance from i_{diff} to the next town of OPT' is at most R .

Now we will consider the remaining distances in OPT' . Every town in OPT' up to and including i_{diff} , is in ALG, and every town in OPT' after i_{diff} is in OPT. Since ALG and OPT are both valid solutions, all of these remaining distances are at most R . Lastly, OPT' has the same number of towns as OPT, so OPT' is an optimal solution.

+2 Correct description for j

+2 Correct explanation for $j < i_{\text{diff}}$

+2 Correct explanation that all distances before and into i_{diff} are at most R

+2 Correct explanation that the distance from i_{diff} to the next town is at most R , and that all remaining distances are valid

+2 Correct explanation for optimality of OPT'

- (4 pts) (c) Explain in your own words why the previous part (proving that OPT' is an optimal solution that contains $i_1, \dots, i_{\text{diff}}$) ultimately proves that your algorithm is correct.

Solution: The “first difference” point between ALG and OPT' (if there is one) is later than that of ALG and OPT. Specifically, OPT contains all of $i_1, i_2, i_3, \dots, i_{\text{diff}-1}$, but not i_{diff} , while OPT' contains all of $i_1, i_2, i_3, \dots, i_{\text{diff}}$. This exchange argument can be applied repeatedly (or inductively), pushing the “first difference” point later and later until we get an optimal solution that has no difference point, i.e., it is identical to ALG, and hence ALG itself is optimal.

Formally, the induction argument is as follows. Our goal is to prove the following statement: for all $0 \leq j \leq k$, there exists an optimal solution containing the first j towns chosen by ALG. After achieving this goal, we set $j = k$ to be the total number of towns chosen by ALG, to conclude that there exists an optimal solution OPT* containing ALG. This means that OPT* = ALG because if OPT* contained any other towns, that would contradict the fact that OPT is an optimal solution (with smallest possible number of towns).

To prove the above statement by induction, the base case $j = 0$ is trivial. The inductive step was proved in the previous part. Specifically, we showed: if there exists an optimal solution OPT containing the first $j = \text{diff} - 1 < k$ towns chosen by ALG, then there exists an optimal solution OPT' containing the first $j + 1 = \text{diff}$ towns chosen by ALG.

+2 States/implies that this swap can be repeated with every difference

+2 States that repeating this swap for every different point gives an optimal solution that is the same as ALG

- (5 pts) (d) Now, suppose that the price of a fuel pack may differ from town to town. For example, a fuel pack might cost \$8 in the first town, \$3 in the second town, etc. So, the input now also includes an array $C = [c_1, \dots, c_n]$, where c_i is the cost of refueling in the i th town. The goal now is to complete the tour while *minimizing the total cost* of the fuel packs that are purchased along the way.

Michael claims that we should use the same greedy algorithm that you proposed above to solve this problem. Determine whether this claim is true or not.

- If it is true, give a proof using a greedy exchange argument.
- Otherwise, provide a small counterexample: give a specific input, the output of your greedy algorithm on that input, and a better solution for that input; also, *very briefly* explain where your proof from the previous part fails for this problem.

Solution: The claim is not true. Let $D = [1, 1, 1]$, $R = 2$, and $C = [1, 2000, 1]$. The greedy algorithm will refuel (only) at town 2, at a cost of \$2000, but we could have refueled at town 1 and finished at the final town (without any more refueling) at a total cost of \$1.

The proof in the previous part fails at the sentence “Lastly, OPT' has the same number of towns as OPT , so OPT' is an optimal solution.” This is no longer true because the cost of OPT' could be worse than that of OPT . Specifically, we formed OPT' from OPT , exchanging out town j for i_{diff} , but the cost at town j could be less than the cost at town i_{diff} .

- +1 State that the algorithm is not optimal
- +2 Valid counterexample
- +2 Correct explanation for the counterexample

4. Dynamic programming for dynamic shortest paths.

Graphs that arise in real applications are often not fixed, but can change over time; such graphs are called “dynamic.” (This is a totally different use of the word “dynamic” than in “dynamic programming”. Computer scientists sometimes name things in confusing ways!) For example, roads and intersections can be added to or deleted from the road network, computers can be added to or removed from the Internet, and people can (un)follow each other on social networks. For many problems of interest, there are algorithms for dynamic graphs that are faster than just re-computing answers “from scratch” whenever the graph changes. In this problem you will give such algorithms for shortest-path problems.

Let $G = (V, E)$ be a weighted directed graph with $n = |V|$ vertices, where the weight of each edge (u, v) is denoted $w(u, v)$. (There can be negative-weight edges, but there is no negative-weight cycle in G .) Suppose that we have already computed the all-pairs distance table D of G . That is, for each vertex pair $u, v \in V$, entry $D(u, v)$ stores the distance (i.e., the length of a shortest path) from u to v in G .

Now, a new vertex v_{new} is added to the graph, together with its incident edges. Let $G_{\text{new}} = (V \cup \{v_{\text{new}}\}, E \cup E_{\text{new}})$ denote the updated graph, where E_{new} consists of all the incoming and outgoing edges for v_{new} . (There is no negative-weight cycle in G_{new} .) We aim to compute the updated distance table D_{new} of G_{new} .

A straightforward approach is to compute D_{new} from scratch using the Floyd-Warshall algorithm, in $\Theta(n^3)$ time. But we want to do better by exploiting the fact that we already have D . In this problem, you will obtain a faster $O(n^2)$ -time algorithm.

- (10 pts) (a) Write expressions for $D_{\text{new}}(v_{\text{new}}, u)$ and $D_{\text{new}}(u, v_{\text{new}})$ that hold for all $u \in V$, where the expressions are in terms of the already-known quantities $D(y, z)$ for $y, z \in V$, and $w(y, z)$ for $(y, z) \in E \cup E_{\text{new}}$. Evaluating your expressions should take $O(n)$ time for each vertex u , for a total of $O(n^2)$ time. Justify the correctness of your expressions and the evaluation time.

Hint: Consider why the Bellman-Ford algorithm is correct.

Solution: For each $u \in V$, we claim that

$$D_{new}(v_{new}, u) = \min_{(v_{new}, w) \in E_{new}} (w(v_{new}, w) + D(w, u)).$$

Observe that this expression minimizes over all of the edges $(v_{new}, w) \in E_{new}$, of which there are at most n (out of up to $2n$ total in E_{new}). So, it takes $O(n)$ time to compute each $D_{new}(v_{new}, u)$, as needed.

For correctness, recall that any shortest path P from v_{new} to u in G_{new} must be a concatenation of some edge $(v_{new}, w) \in E_{new}$, followed by a shortest path P' from w to u in G_{new} . We claim that, without loss of generality, P' is actually a shortest path from w to u in the *original* graph G . This is because there is no negative-weight cycle, path P does not have any cycle (w.l.o.g.), so P' does not visit v_{new} . Hence P' uses only original edges in G , and therefore is indeed a shortest path in G . The above expression minimizes the cost over all possible ways a path could have the above-described form, and therefore it yields the distance from v_{new} to u .

Symmetrically, for each $u \in V$, we have that

$$D_{new}(u, v_{new}) = \min_{(w, v_{new}) \in E_{new}} (D(u, w) + w(w, v_{new})).$$

The running time and correctness analysis are symmetric too.

+2 Correct expression for $D_{new}(v_{new}, u)$.

+2 Correctness analysis for it.

+1 Runtime analysis for it.

+2 Correct expression for $D_{new}(u, v_{new})$.

+2 Correctness analysis for it.

+1 Runtime analysis for it.

Note: If for the correctness and runtime of the second expression, the proof for the first one is cited and claimed to be similar (as is done in the solution above), award full points.

- (8 pts) (b) Write an expression for $D_{new}(u, v)$ that holds for all $u, v \in V$, where the expression is in terms of the already-known quantities listed in the previous part, as well as the quantities you computed in the previous part. Evaluating your expression for all $u, v \in V$ should take a total of $O(n^2)$ time. Justify the correctness of your expression and the evaluation time. Observe that by combining the two parts, we have computed the entire new distance table D_{new} of G_{new} in $O(n^2)$ time!
- Hint:* Consider why the Floyd-Warshall algorithm is correct.

Solution: For all $u, v \in V$, we claim that

$$D_{\text{new}}(u, v) = \min\{D(u, v), D_{\text{new}}(u, v_{\text{new}}) + D_{\text{new}}(v_{\text{new}}, v)\}.$$

We can compute any such value in $O(1)$ time, so the overall running time is $O(n^2)$ (because there are $O(n^2)$ pairs u, v). Observe that this corresponds to just one (more) iteration of the “outer loop” of Floyd-Warshall, where we are newly allowing v_{new} as an intermediate vertex on paths.

The correctness proof is also the same as that of the Floyd-Warshall algorithm. We know that a shortest path from u to v in G_{new} either has v_{new} as an intermediate vertex, or not. If so, their distance is $D_{\text{new}}(u, v_{\text{new}}) + D_{\text{new}}(v_{\text{new}}, v)$. If not, the distance is $D(u, v)$ because G is exactly the graph G_{new} after ignoring v_{new} . The above expression captures the best cost between these two possibilities.

+3 Correct expression for $D_{\text{new}}(u, v)$.

+3 Correctness analysis.

+2 Runtime analysis.

5. MST mashup.

- (5 pts) (a) Consider the following claim: a minimum spanning tree in a graph is *unique* if the edge weights are *distinct* (all different).

Professor Υ presents you with the following bogus “proof” of this claim: *because edge weights are distinct, all choices in Kruskal’s algorithm are completely determined—there are no “ties” between edge weights that the algorithm can choose how to break. So, the algorithm has only one possible output. Because Kruskal’s algorithm is correct for the MST problem, its unique output must be the only minimum spanning tree in the graph.*

Briefly explain what is wrong with this “proof.” In particular, indicate which sentence in the “proof” does not follow from the previous one, and provide a reason for it.

Solution: The last sentence does not logically follow from the previous ones. We have not ruled out the possibility that there are other MSTs that Kruskal’s algorithm cannot output (regardless of what choices it makes). More generally, even if the output of some specific algorithm (Kruskal’s algorithm in this case) is uniquely determined, this does not imply that this output is only optimal solution.

As an concrete example to illustrate the previous point (not required for full points), consider the “back to college” problem on this homework. We’ll exhibit an input for which the greedy algorithm must output a specific unique solution, but a different optimal solution also exists. Let the input distances be $d_1 = d_2 = d_3 = 1$, and let $R = 2$. There are no arbitrary choices for the algorithm to make, only greedy ones, and the greedy choice is to refuel at town 2 alone (after traveling a distance of $R = 2$). But this is not the only optimal solution; it is also optimal to refuel at town 1 alone (after traveling distance 1) and then travel the remaining distance of 2 to complete the tour.

- +2 Identify the last sentence is where the problem lies (which often be implicit via the reasoning provided)
- +3 Mention that Kruskal outputting a unique MST doesn't imply that's the *only* MST

(8 pts)

- (b) Give a correct proof for the claim in the previous part.

Hint: Consider two distinct spanning trees T_1, T_2 and prove via an exchange argument that one of them does not have minimum weight.

Solution: Let T_1 and T_2 be any two distinct spanning trees of the graph. We will show that at least one of them is *not* a *minimum* spanning tree, using an exchange argument very similar to the one from the correctness proof for Kruskal's algorithm. This implies the claim, that there cannot be two different minimum spanning trees.

Let e be the unique lightest edge that is in only one of the two spanning trees, which is T_1 without loss of generality. (Such an e must exist because the spanning trees are different, and e is unique because the edge weights are distinct.) As in the correctness proof for Kruskal's algorithm, $T_2 \cup \{e\}$ contains a cycle C . Because T_1 doesn't have a cycle, C (and hence T_2) must have an edge $f \notin T_1$, and because $e \in T_1$ is the lightest edge in exactly one of the trees, $w(e) < w(f)$. Then $T'_2 := T_2 \cup \{e\} \setminus \{f\}$ is a spanning tree that is strictly lighter than T_2 . So T_2 is not a minimum spanning tree, as needed.

Alternative solution: We can also prove the claim in a different way, using a proof by contradiction via the correctness argument for Kruskal's algorithm. Let K be the output of Kruskal's algorithm, which is fully determined by (the correct part of) the argument from the previous part, and which is an MST of the graph.

Now assume for the purpose of contradiction that there is some *other* MST $T \neq K$ of the graph. So, there is some first edge $e \in K$ chosen by the algorithm that is not in T . By the exchange argument for Kruskal's algorithm, there is some edge $f \in T \setminus K$ such that $w(e) < w(f)$ (the inequality is strict because edge weights are distinct) and $T' = T \cup \{e\} \setminus \{f\}$ is a spanning tree. But this means that $w(T') < w(T)$, which contradicts the hypothesis that T is a *minimum-weight* spanning tree. Therefore, our original assumption (that there is some MST different from K) is false, and hence K is the unique MST of the graph.

- +3 Attempt an exchange argument or proof by contradiction with two spanning trees
- +5 Correct reasoning

6. Greedy knapsack filling.

Recall the knapsack problem (sometimes called the “0-1” knapsack problem): you are given n items, where the i th item has weight w_i and value v_i (both positive integers), and a non-negative weight capacity C of the knapsack. An optimal solution is a subset $S \subseteq \{1, 2, \dots, n\}$ of the items having maximum total value

$$\sum_{i \in S} v_i,$$

under the constraint that the total weight is at most the knapsack capacity, i.e.,

$$\sum_{i \in S} w_i \leq C.$$

In this problem, we introduce the *fractional* variant of the knapsack problem, where one may take any fraction $p_i \in [0, 1]$ of any item i . Naturally, a p_i -fraction of item i weighs $p_i \cdot w_i$, and has value $p_i \cdot v_i$. The goal is to maximize the total value of the selected fractional items.

- (2 pts) (a) Briefly explain why the optimal value for the fractional knapsack problem is *at least as large* as that of the original 0-1 knapsack problem (for the same weights, values, and knapsack capacity).

Solution: Every valid choice of items in the 0-1 knapsack problem is also a valid choice in the fractional knapsack problem. So, the optimal total value for the fractional problem is at least as large as the optimal value for the 0-1 problem.

+2 Mention that a valid set of choices for the 0-1 knapsack problem is also valid for the fractional knapsack problem

- (5 pts) (b) Consider the following two greedy algorithms for the fractional knapsack problem:
- **VALUEGREEDY:** While there is still some unused knapsack capacity and at least one unconsidered item, choose an item having the *largest value* (among those not considered yet), and add the largest possible fraction of that item (up to the entire item) that will fit within the remaining knapsack capacity.
 - **WEIGHTGREEDY:** While there is still some unused knapsack capacity and at least one unconsidered item, choose an item having the *smallest weight* (among those not considered yet), and add the largest possible fraction of that item (up to the entire item) that will fit within the remaining knapsack capacity.

Suppose we run *both* algorithms VALUEGREEDY and WEIGHTGREEDY, and output a best one of their two outputs. Is this a correct algorithm for the fractional knapsack problem? If so, prove it. Otherwise, give an input for which this algorithm's output is not optimal, and show why it is not.

Solution: The algorithm is not correct. To construct a counterexample, the basic observation is that VALUEGREEDY chooses high-value items without regard to their weights, and WEIGHTGREEDY chooses low-weight items without regard to their values. So, the counterexample will have some high-value item with large weight, and some low-weight item with tiny value, whereas an optimal solution will consist of an item that is “in the middle.”

Specifically, consider a knapsack with capacity $C = 10$, a ‘high’ item with value 100 and weight 100, a ‘low’ item with value 10 and weight 10, and a ‘medium’ item with value 80 and weight 20. VALUEGREEDY takes just 1/10 of the ‘high’ item, and gets just value 10 from it. WEIGHTGREEDY takes the entire ‘low’ item (and nothing else), and also gets value 10 from it. However, an optimal solution is to take 1/2 of the ‘medium’ item, for a much better value of 40.

(We can tweak the numbers so that an optimal solution is better than the algorithms' outputs by as large of a factor as we like. So, the algorithm in question produces very poor outputs in general.)

- +1 Claim or imply that the algorithm isn't optimal
- +2 Give inputs on which the algorithm performs worse than optimal
- +2 Explanation of why the given algorithm isn't optimal on its input

(10 pts) (c) Consider the following greedy algorithm for the fractional knapsack problem:

- RELATIVELYGREEDY: While there is still some unused knapsack capacity, choose an item having the largest *relative value* v_i/w_i (among those not considered yet), and add the largest possible fraction of that item (up to the full item) that will fit within the remaining knapsack capacity.

In this part, you will prove that the RELATIVELYGREEDY algorithm is a correct algorithm for the fractional knapsack problem. This is notable because we used a more complicated dynamic programming algorithm to solve 0-1 Knapsack, but the fractional variant admits a much simpler greedy algorithm!

The setup for your correctness proof is as follows. Without loss of generality, suppose that the items are in sorted order by relative value $r_i = v_i/w_i$, from largest to smallest (the algorithm considers them in this order). Let $\text{ALG} = p_1, p_2, \dots, p_n$ be the fractions of items $1, 2, \dots, n$ chosen by the algorithm, respectively. (So, $p_i = 1$ if the algorithm chooses all of item i , and $p_i = 0$ if it doesn't choose any of item i .)

Let OPT be an arbitrary optimal solution. If $\text{ALG} = \text{OPT}$, then ALG is optimal, as needed. So suppose that $\text{ALG} \neq \text{OPT}$, and consider the smallest index j such that OPT does not have *exactly* a p_j -fraction of item j .

You will construct another solution OPT' whose fractions of the first j items are exactly $p_1, p_2, \dots, p_j$ by modifying OPT , using an exchange argument.

Describe which fraction(s) of item(s) to exchange, and prove that OPT' is an optimal solution; this should include a proof that OPT' is a *valid* solution.

Solution:

1. Prove that the fraction of item j included in OPT (denoted by $p_{j\text{OPT}}$) is less than p_j .
2. Prove that both solutions ALG and OPT have the same total weight, and use this to argue that OPT has at least $(p_j - p_{j\text{OPT}}) \cdot w_j$ weight on items $> j$.
3. Create OPT' from OPT by adding $(p_j - p_{j\text{OPT}})$ fraction of item j , thus increasing the weight by $(p_j - p_{j\text{OPT}}) \cdot w_j$, and removing $(p_j - p_{j\text{OPT}}) \cdot w_j$ weight from items $> j$.
4. Argue that value of OPT' is greater than or equal to OPT .

First, we will prove the claim that the fraction of item j included in OPT is less than p_j . The algorithm RELATIVELYGREEDY takes an item of largest relative value and adds the *largest* possible fraction of it to the knapsack. This means that every item with a nonzero fraction has fraction 1 (i.e., the entire item is added), except possibly the last item chosen by the algorithm. Let item k be the last one chosen by ALG. So we have proved the claim if $j < k$. The claim is also true if $j = k$ itself, because the algorithm adds the *largest* fraction of item k that fits into the knapsack after having taken the entirety of items 1 through $k - 1$, and OPT also has all of these items. Finally, it cannot be the case that $j > k$, because ALG does not take any part of the items greater than k because there is no remaining capacity for them, and hence neither can OPT (because it matches ALG on all items from 1 through $j - 1 \geq k$).

Next, we prove that both solutions ALG and OPT have the same total weight. There are two cases: If the total weight of all items is less than the capacity of the knapsack, then both ALG and OPT will fit every item in its entirety into the knapsack. On the other hand, if the total weight of all items exceeds the capacity of the knapsack, both ALG and OPT will use the entire weight capacity of the knapsack. Thus, ALG and OPT have the same total weight.

Let $p_{j\text{OPT}}$ be the fraction of item j included in OPT. To form OPT' from OPT, we add a $p_j - p_{j\text{OPT}} > 0$ fraction of item j , and we need to remove an *equal weight* of other item(s) $> j$ to ensure that OPT' is within the total weight capacity. The amount of added weight that must be removed is $w = (p_j - p_{j\text{OPT}}) \cdot w_j$.

By hypothesis, OPT includes all items $< j$ with fraction 1 (as does ALG), and includes item j with fraction $p_{j\text{OPT}} < p_j$. Because the total weights of ALG and OPT are equal, at least w of the weight in OPT is occupied by items $> j$. So, we can indeed remove total weight w from items $> j$ in some arbitrary fashion, possibly spread over multiple items (since there might not be a single partial item in OPT of weight at least w).

Now we will argue that the value of OPT' is at least the value of OPT, so OPT' is indeed an optimal solution. To form OPT', we started with OPT, added weight w of item j , and removed weight w from item(s) $> j$, which have relative value at most v_j/w_j . We will calculate the total value added and removed. The value added is $(p_j - p_{j\text{OPT}}) \cdot v_j$. The value removed is at most the weight w times the relative-value upper bound v_j/w_j : this is $(p_j - p_{j\text{OPT}}) \cdot w_j \cdot (v_j/w_j) = (p_j - p_{j\text{OPT}}) \cdot v_j$. This is true because the relative value of all item(s) $> j$ is lesser than or equal to v_j/w_j . Thus, the total value removed is at most the total value added, so the value of OPT' is at least the value of OPT, as needed.

Alternative Solution: Another solution involves adding the same $(p_j - p_{j\text{OPT}})$ fraction to item j , but after, instead of deleting an *equal weight* from item(s) $> j$, we will delete an *equal value* of item(s) $> j$. This will ensure that the value of OPT' equals the value of OPT. Then, we show that the weight of the removed items is at least $(p_j - p_{j\text{OPT}}) \cdot w_j$ (the added weight), and thus, the weight of OPT' is at most weight of OPT, which is at most C , because OPT is a valid (optimal) solution by assumption. Therefore, OPT' is another optimal, valid solution.

First, we need to argue that total value of the item(s) $> j$ is at least $(p_j - p_{j\text{OPT}}) \cdot v_j$. This has to be true, otherwise the value of ALG is more than the value of OPT,

which is not true. So we can indeed reduce the fractions of item(s) $> j$ such that the value decreases by $(p_j - p_{j\text{OPT}}) \cdot v_j$. Now we will calculate the weight added and removed. The weight added is $(p_j - p_{j\text{OPT}}) \cdot w_j$. The weight removed is at least $(p_j - p_{j\text{OPT}}) \cdot v_j \cdot (w_j/v_j)$, because the relative value of item(s) $> j$ is lesser than or equal to that of j (that is, $(w_k/v_k) \geq (w_j/v_j)$ for all $k \geq j$). Therefore the weight removed is greater than or equal to the weight added, and hence the weight of OPT' is lesser than or equal to weight of OPT (which in turn is at most C). We already proved that the values of OPT' and OPT are equal, and hence OPT' is optimal as well.

- +3 Attempt a proof that OPT includes $\leq p_j$ of item j
- +2 Correct description of OPT'
- +2 Correct explanation for validity of OPT'
- +3 Correct explanation for optimality of OPT'

7. Optional extra credit: Greedily independent.

An *independent set* of an undirected graph $G = (V, E)$ is a subset of vertices $S \subseteq V$ such that no two vertices in S are adjacent, i.e., $(u, v) \notin E$ for every distinct $u, v \in S$. We are interested in a *maximum independent set*, i.e., the independent set with the *largest* number of vertices. (There may be more than one maximum independent set.)

Consider the following greedy algorithm for finding an independent set in a graph.

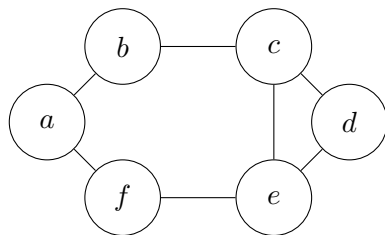
```

1: function GREEDYIS( $G = (V, E)$ )
2:    $S \leftarrow \emptyset$ 
3:   while  $G$  has at least one vertex do
4:     Select any vertex  $v$  in  $G$  that has smallest degree (i.e., the fewest neighbors)
5:     Add  $v$  to  $S$ 
6:     Remove  $v$  and all its neighbors, including all their incident edges, from  $G$ 
7:   return  $S$ 

```

(1 EC pt)

- (a) Give an example of a graph for which the algorithm GREEDYIS **does not** return the maximum independent set. You can assign numbers to the vertices of your graph, and assume that in the case where there are multiple vertices with the smallest degree, GREEDYIS will choose the vertex having the smallest number.



Solution:

The algorithm outputs $S = \{a, c\}$. This is not optimal: $\{b, d, f\}$ is a larger independent set.

+1 Example is provided, independent set returned by GREEDYIS (I) is specified, maximum independent set (I') is specified, and I, I' are not equal.

(2 EC pts)

- (b) In this part, you will prove that GREEDYIS returns a maximum independent set when the input graph is a *tree*.

The setup for the proof is as follows. Let $\text{ALG} = v_1, v_2, v_3, \dots, v_k$ be the vertices chosen by your algorithm, in their greedily chosen order, and let OPT be some arbitrary optimal solution. If $\text{ALG} = \text{OPT}$, then ALG is optimal, as needed. So suppose that $\text{ALG} \neq \text{OPT}$, and consider the *first* vertex v_{diff} in ALG that is not in OPT . (Such a v_{diff} must exist because ALG cannot be a proper subset of OPT ; do you see why?) So, OPT contains all of $v_1, v_2, v_3, \dots, v_{\text{diff}-1}$, but not v_{diff} .

You will construct another optimal solution (a maximum independent set) OPT' that has all of $v_1, v_2, v_3, \dots, v_{\text{diff}}$ by modifying OPT , exchanging out some suitable vertex u for v_{diff} instead.

Describe which vertex u to exchange for v_{diff} , and prove that OPT' is both valid *and* optimal.

Solution: Throughout the entire algorithm, the graph is a forest, or collection of trees (since any subgraph of a tree is a forest). Let F_{diff} be the state of the graph (forest) at the point in time that the algorithm chooses v_{diff} . Since any forest contains at least one vertex of degree ≤ 1 , and the algorithm chooses a vertex of minimum degree, we know that v_{diff} has degree ≤ 1 in the subgraph F_{diff} .

Let $u \neq v_{\text{diff}}$ be the unique neighbor of v_{diff} in F_{diff} , if such a neighbor exists.

We construct OPT' from OPT by exchanging out u for v_{diff} , i.e., $\text{OPT}' = \text{OPT} \cup \{v_{\text{diff}}\} \setminus \{u\}$. (In case v_{diff} has no neighbor, define OPT' as simply $\text{OPT} + v_{\text{diff}}$. However, below we show that OPT' is an independent set of the full tree, so in fact such a neighbor must exist—otherwise, OPT' would be larger than OPT , which would contradict the fact that OPT is a *maximum* independent set.)

To show that OPT' is an independent set, it suffices to show that no neighbor of v_{diff} in the original tree is in OPT' . (This suffices because the other elements of OPT' are in OPT , and no pair of them has an edge between them.) The only possible neighbor of v_{diff} in F_{diff} is u , and u is not in OPT' by definition.

Now we need to show that OPT' contains no neighbors of v_{diff} that are not in F_{diff} . Now let W be the set of neighbors of v_{diff} that are not in F_{diff} . Our goal is to show that every element of W is *not* in OPT (and thus not in OPT').

Claim 1. For each vertex $w \in W$, ALG and OPT agree on whether or not w is in the independent set.

The consequence of this claim is that since no element of W is in ALG (otherwise ALG wouldn't be a valid independent set), no element of W is in OPT either.

Proof of claim: No $w \in W$ is in $\text{ALG} \setminus \text{OPT}$, as otherwise the “first difference” point would have been earlier. Additionally, no $w \in W$ is in $\text{OPT} \setminus \text{ALG}$ because by the

definition of the algorithm, every vertex not in F_{diff} was removed at some point before v_{diff} was chosen. Also by the definition of the algorithm, each removed vertex is either added to ALG (and thus OPT) or adjacent to a vertex added to ALG (and thus OPT). So, if there were such a $w \in W$ in $\text{OPT} \setminus \text{ALG}$, this would mean that OPT would not be a valid independent set.

This completes the proof of the claim, and thus the proof that OPT' is indeed an independent set.

Lastly, OPT' is a *maximum* independent set because it contains at least as many vertices as OPT, which itself is a maximum independent set.

+1 Use the fact that v_{diff} must have a unique neighbor.

+1 Correctly prove that OPT' is an independent set, and in particular, a maximum independent set (that is, it contains as many vertices as OPT).